SHETH L.U.J AND SIR M.V. COLLEGE

SUBJECT: CYBER SECURITY

PRACTICAL - 1

AIM: Implementing Substitution and Transposition Ciphers.

1.A Design and implement algorithms to encrypt and decrypt messages using classical substitution techniques

1. B Design and implement algorithms to encrypt and decrypt messages using transposition techniques

1.A

CLI:

CODE:

```
# Practical 1.A
import string

#Caesar Cipher
def caesar_encrypt(text, key):
    result = ""

    for ch in text:
        if ch.isalpha():
            if ch.isupper():
                result += chr((ord(ch) - 65 + key) % 26 + 65)
            else:
                result += chr((ord(ch) - 97 + key) % 26 + 97)
        else:
            result += ch

    return result


def caesar_decrypt(text, key):
    return caesar_encrypt(text, -key)


#Monoalphabetic Cipher
alphabet = string.ascii_uppercase
key = "QWERTYUIOPASDFGHJKLZXCVBNM"

encrypt_dict = {}
decrypt_dict = {}

for i in range(26):
    encrypt_dict[alphabet[i]] = key[i]
    decrypt_dict[key[i]] = alphabet[i]
```


T125 - SUMEET

```
def mono_encrypt(text):
    result = ""

    for ch in text.upper():
        if ch in encrypt_dict:
            result += encrypt_dict[ch]
        else:
            result += ch

    return result


def mono_decrypt(text):
    result = ""

    for ch in text.upper():
        if ch in decrypt_dict:
            result += decrypt_dict[ch]
        else:
            result += ch

    return result


#Main Menu
while True:

    print("\n===== Classical Substitution Techniques =====")
    print("1. Caesar Cipher")
    print("2. Monoalphabetic Cipher")
    print("3. Exit")

    choice = input("Enter Choice: ")

    if choice == "1":

        print("\n1. Encrypt")
        print("2. Decrypt")

        op = input("Enter Choice: ")

        message = input("Enter Message: ")
```


T125_SUMEET

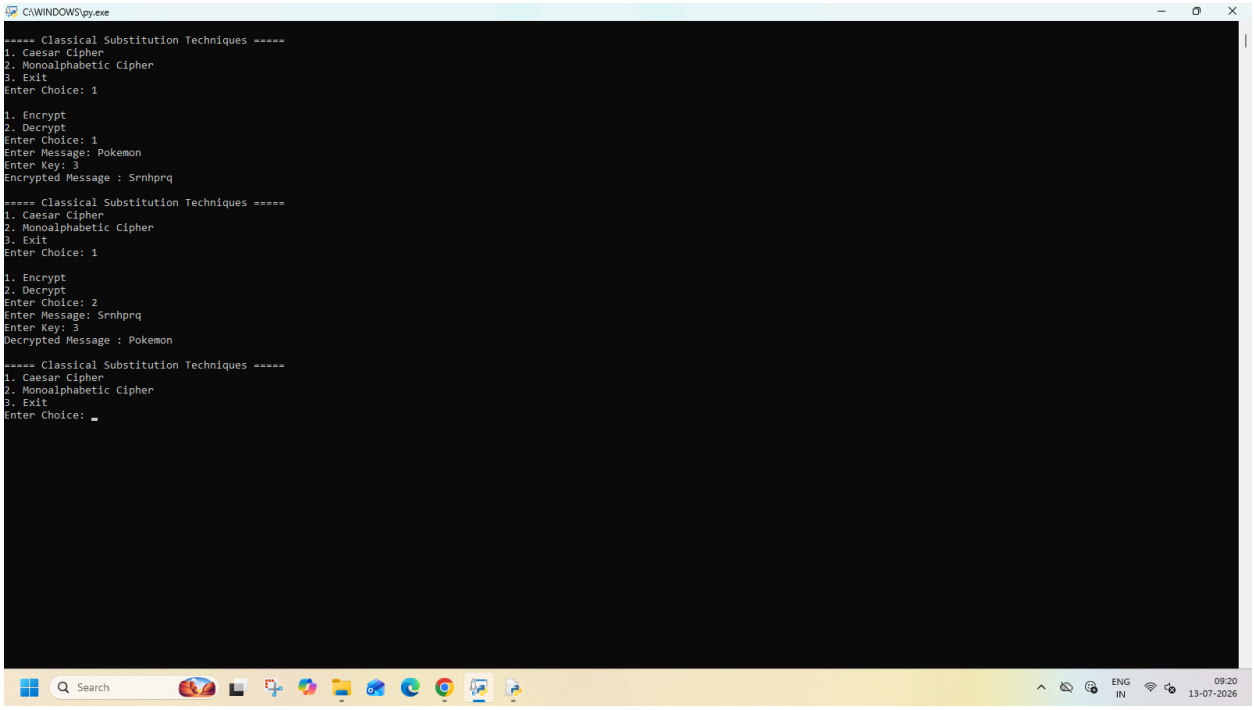
```
    shift = int(input("Enter Key: "))

    if op == "1":
        print("Encrypted Message :", caesar_encrypt(message, shift))
    elif op == "2":
        print("Decrypted Message :", caesar_decrypt(message, shift))
    else:
        print("Invalid Choice")

elif choice == "2":

    print("\n1. Encrypt")
    print("2. Decrypt")

    op = input("Enter Choice: ")

    message = input("Enter Message: ")

    if op == "1":
        print("Encrypted Message :", mono_encrypt(message))
    elif op == "2":
        print("Decrypted Message :", mono_decrypt(message))
    else:
        print("Invalid Choice")

elif choice == "3":
    print("Program Ended")
    break

else:
    print("Invalid Choice")
```

T125 _ SUMEET

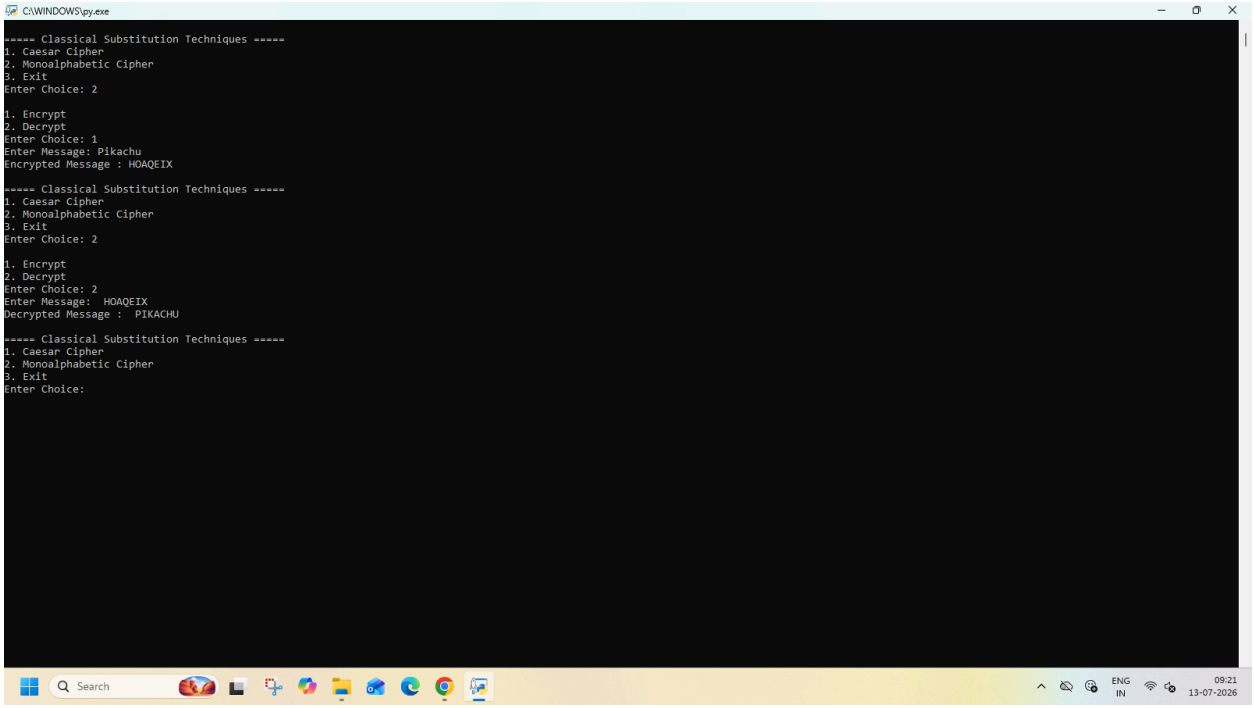
SHETH L.U.J AND SIR M.V. COLLEGE
SUBJECT: CYBER SECURITY

Output:

1.Caesar Cipher:



2.Monoalphabetic Cipher:



SUMEET YADAV
T125

GUI:

CODE:

```
# Practical 1.A GUI
import tkinter as tk
from tkinter import ttk
import string

#Caesar Cipher
def caesar_encrypt(text, key):
    result = ""

    for ch in text:
        if ch.isalpha():
            if ch.isupper():
                result += chr((ord(ch) - 65 + key) % 26 + 65)
            else:
                result += chr((ord(ch) - 97 + key) % 26 + 97)
        else:
            result += ch

    return result


def caesar_decrypt(text, key):
    return caesar_encrypt(text, -key)


#Monoalphabetic Cipher
alphabet = string.ascii_uppercase
key = "QWERTYUIOPASDFGHJKLZXCVBNM"

encrypt_dict = {}
decrypt_dict = {}

for i in range(26):
    encrypt_dict[alphabet[i]] = key[i]
    decrypt_dict[key[i]] = alphabet[i]


def mono_encrypt(text):
    result = ""

    for ch in text.upper():
        if ch in encrypt_dict:
```


T125 SUMEET

```
            result += encrypt_dict[ch]
        else:
            result += ch

    return result


def mono_decrypt(text):
    result = ""

    for ch in text.upper():
        if ch in decrypt_dict:
            result += decrypt_dict[ch]
        else:
            result += ch

    return result


#GUI Functions
def encrypt_message():

    cipher = cipher_choice.get()
    message = message_entry.get()

    if cipher == "Caesar Cipher":
        key = int(key_entry.get())
        result = caesar_encrypt(message, key)

    else:
        result = mono_encrypt(message)

    output.set(result)


def decrypt_message():

    cipher = cipher_choice.get()
    message = message_entry.get()

    if cipher == "Caesar Cipher":
        key = int(key_entry.get())
        result = caesar_decrypt(message, key)
```

T125 - SUMEET

```
    else:
        result = mono_decrypt(message)

    output.set(result)


#GUI Window
root = tk.Tk()
root.title("Classical Substitution Techniques")
root.geometry("500x400")


title = tk.Label(
    root,
    text="Caesar & Monoalphabetic Cipher",
    font=("Arial", 16, "bold")
)
title.pack(pady=10)


# Cipher Selection
tk.Label(root, text="Select Cipher").pack()

cipher_choice = ttk.Combobox(
    root,
    values=[
        "Caesar Cipher",
        "Monoalphabetic Cipher"
    ]
)

cipher_choice.current(0)
cipher_choice.pack()


# Message Input
tk.Label(root, text="Enter Message").pack()

message_entry = tk.Entry(
    root,
    width=45
)

message_entry.pack()
```

T125 _ SUMEET

```
# Key Input
tk.Label(root, text="Enter Key (Caesar only)").pack()

key_entry = tk.Entry(root)
key_entry.pack()


# Buttons
button_frame = tk.Frame(root)
button_frame.pack(pady=20)


encrypt_button = tk.Button(
    button_frame,
    text="Encrypt",
    width=12,
    command=encrypt_message
)

encrypt_button.grid(row=0, column=0, padx=10)

decrypt_button = tk.Button(
    button_frame,
    text="Decrypt",
    width=12,
    command=decrypt_message
)

decrypt_button.grid(row=0, column=1, padx=10)

# Output
tk.Label(root, text="Result").pack()

output = tk.StringVar()

result_entry = tk.Entry(
    root,
    textvariable=output,
    width=45
)
result_entry.pack()
root.mainloop()
```

SUMEET YADAV
T125

T_125_SUMEET

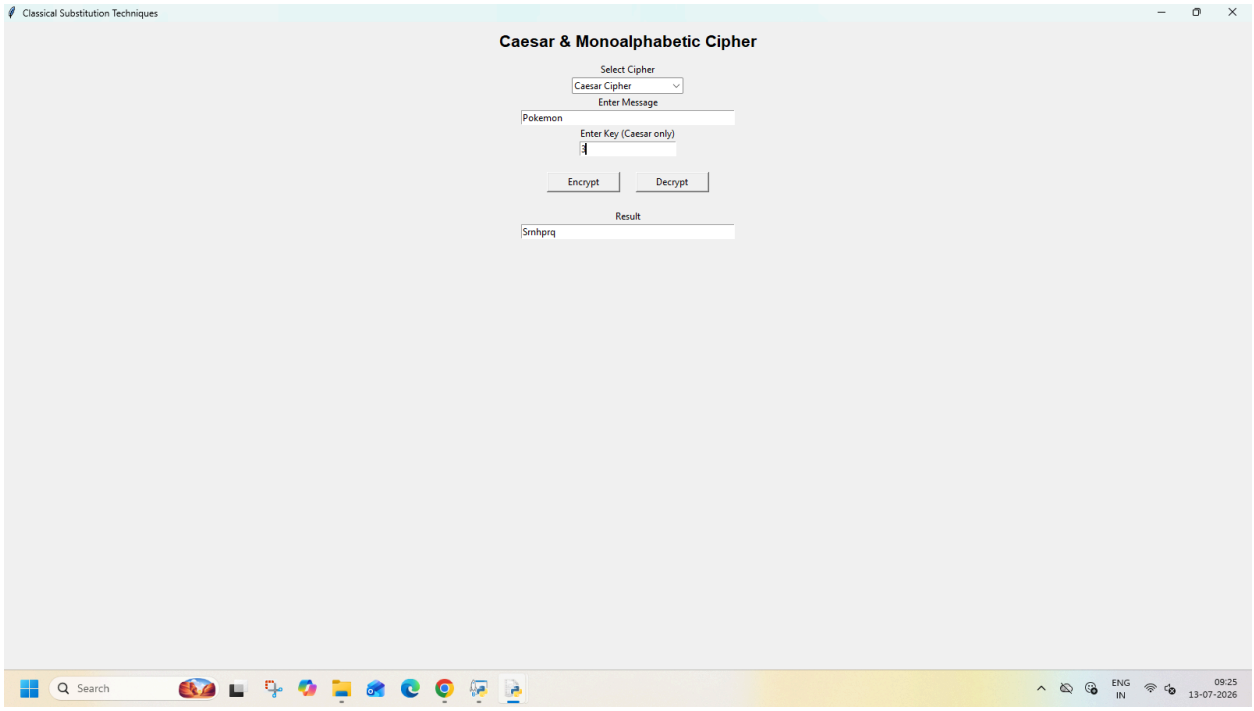
SHETH L.U.J AND SIR M.V. COLLEGE

SUBJECT: CYBER SECURITY

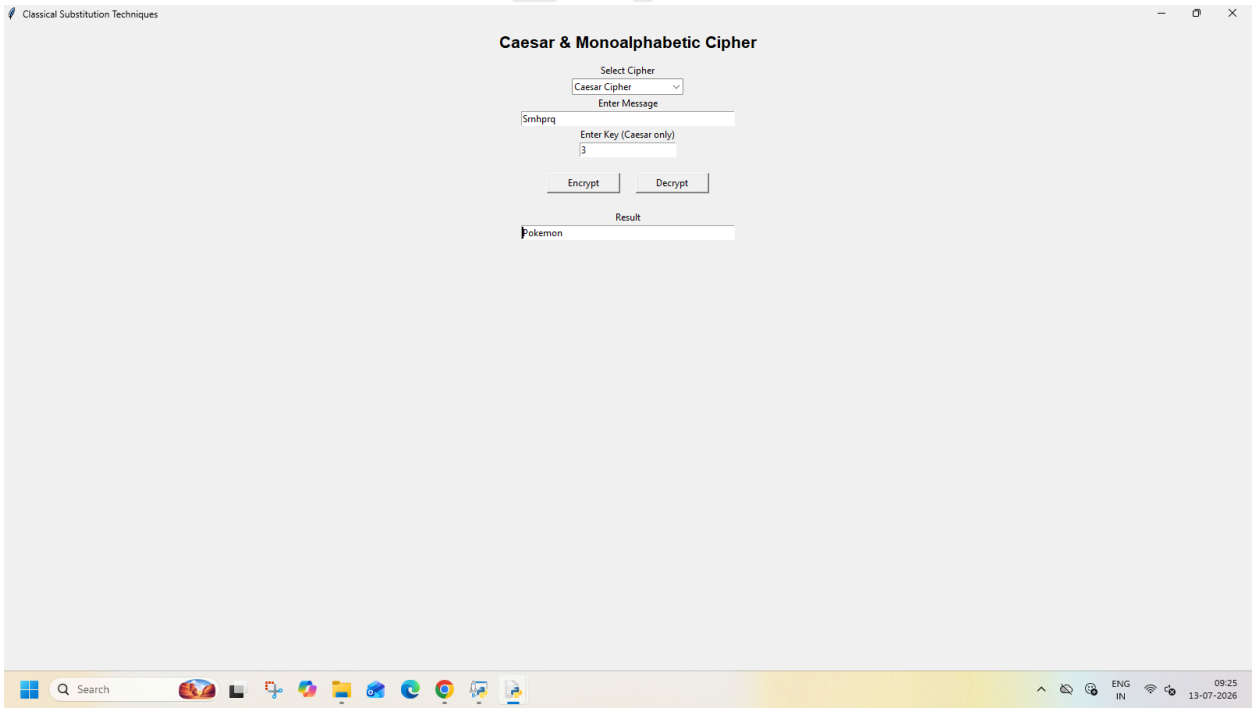
OUTPUT:

Caesar Cipher:

encrypt:



Decrypt:



SUMEET YADAV

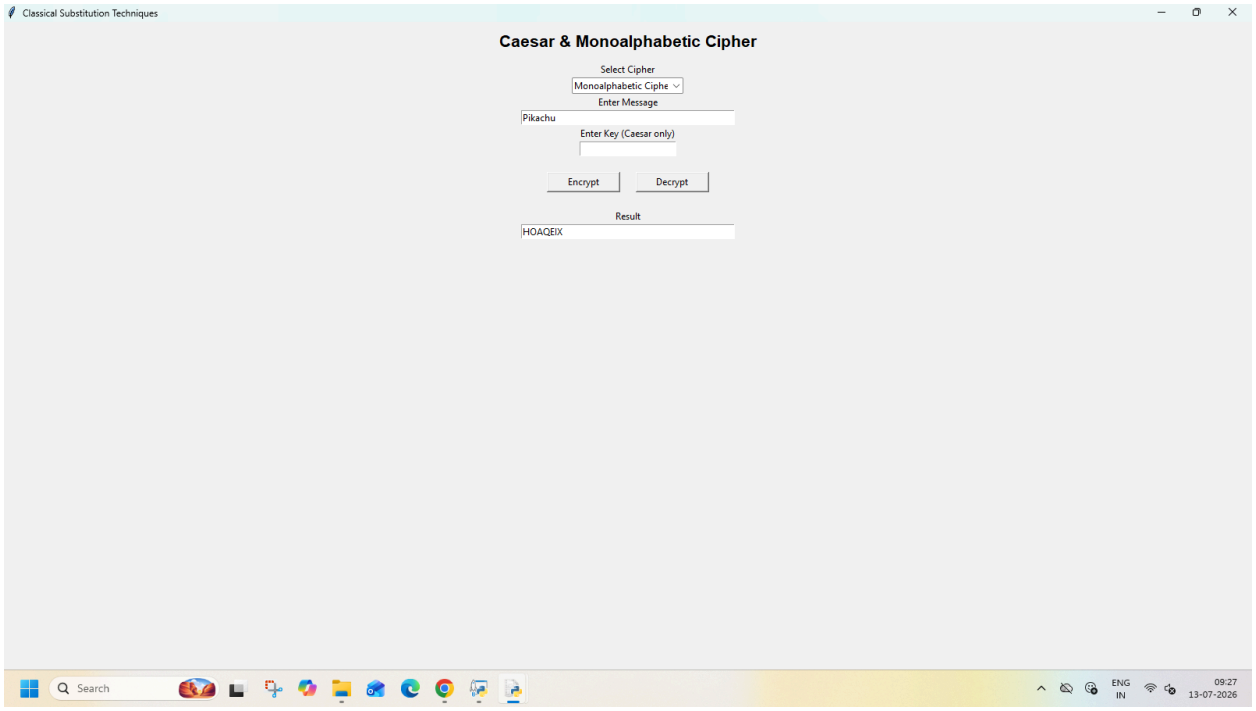
T125

SHETH L.U.J AND SIR M.V. COLLEGE

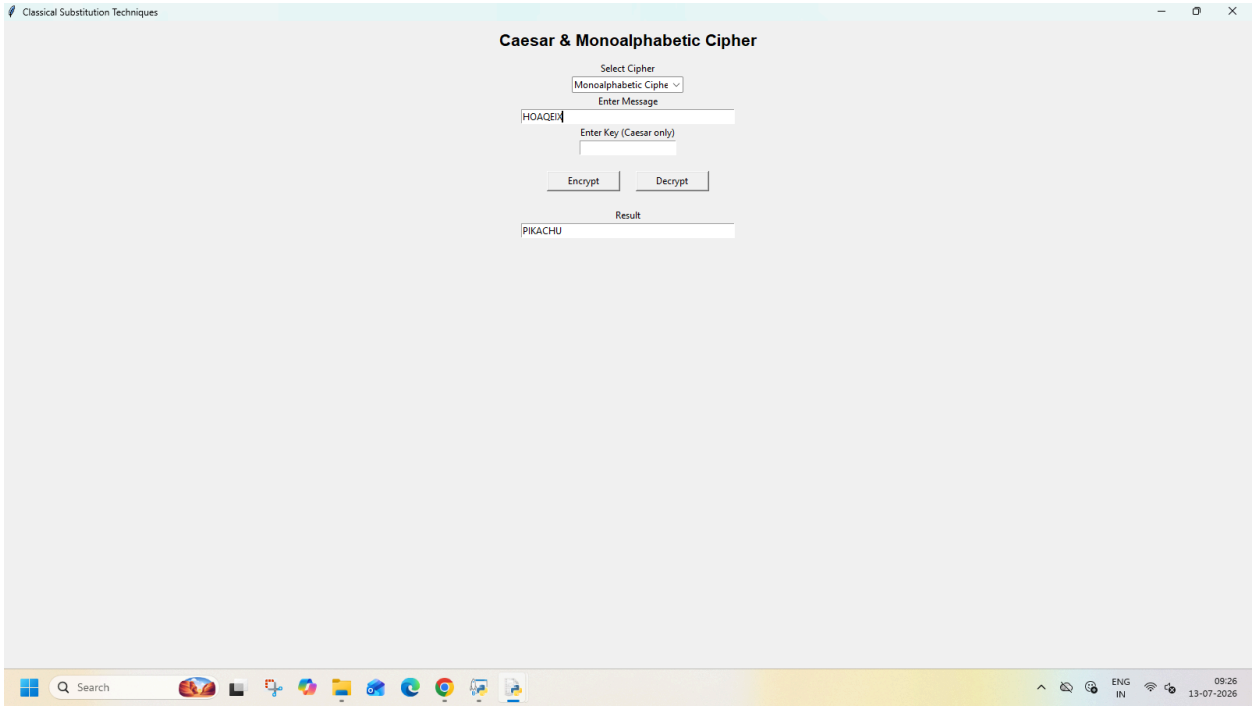
SUBJECT: CYBER SECURITY

Monoalphabetic:

Encrypt:



Decrypt:



SUMEET YADAV

T125

1.B:

CLI:

CODE:

```
import math

#Rail Fence Cipher
def rail_encrypt(text, key):
    if key <= 1:
        return text

    rail = [['\n' for _ in range(len(text))] for _ in range(key)]
    row = 0
    direction = False

    for col in range(len(text)):
        if row == 0 or row == key - 1:
            direction = not direction

        rail[row][col] = text[col]
        row += 1 if direction else -1

    cipher = ""
    for i in range(key):
        for j in range(len(text)):
            if rail[i][j] != '\n':
                cipher += rail[i][j]

    return cipher


def rail_decrypt(cipher, key):
    if key <= 1:
        return cipher

    rail = [['\n' for _ in range(len(cipher))] for _ in range(key)]

    row = 0
    direction = None

    for col in range(len(cipher)):
        if row == 0:
            direction = True
        if row == key - 1:
            direction = False
```


T125_SUMEET

```
        rail[row][col] = '*'
        row += 1 if direction else -1

    index = 0
    for i in range(key):
        for j in range(len(cipher)):
            if rail[i][j] == '*' and index < len(cipher):
                rail[i][j] = cipher[index]
                index += 1

    result = ""
    row = 0
    direction = None

    for col in range(len(cipher)):
        if row == 0:
            direction = True
        if row == key - 1:
            direction = False

        result += rail[row][col]
        row += 1 if direction else -1

    return result


#Columnar Transposition Cipher
def column_encrypt(message, key):
    cipher = [''] * key

    for col in range(key):
        pointer = col
        while pointer < len(message):
            cipher[col] += message[pointer]
            pointer += key

    return ''.join(cipher)


def column_decrypt(cipher, key):
    cols = math.ceil(len(cipher) / key)
    rows = key
    shaded = cols * rows - len(cipher)
```


T125 / SUMEET

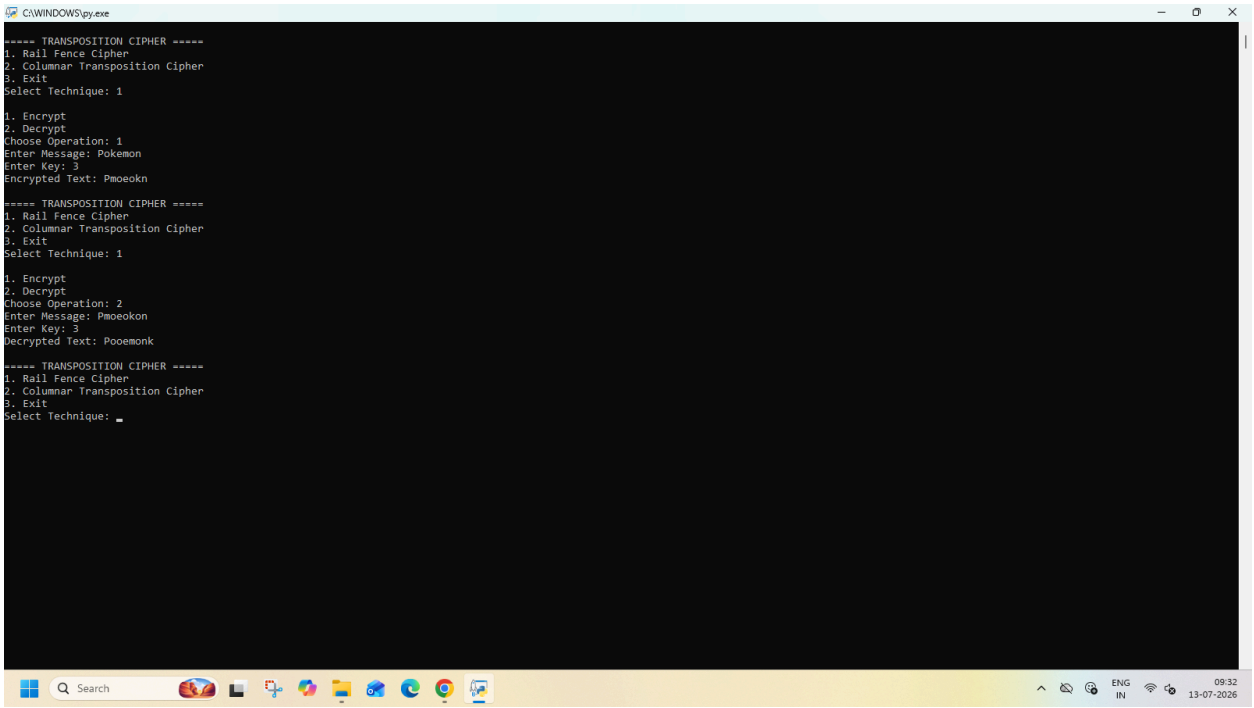
```
    plain = [''] * cols

    col = 0
    row = 0

    for symbol in cipher:
        plain[col] += symbol
        col += 1

        if col == cols or (col == cols - 1 and row >= rows - shaded):
            col = 0
            row += 1

    return ''.join(plain)


#Main Program
while True:
    print("\n===== TRANSPOSITION CIPHER =====")
    print("1. Rail Fence Cipher")
    print("2. Columnar Transposition Cipher")
    print("3. Exit")

    choice = input("Select Technique: ")

    if choice == "3":
        print("Program Ended.")
        break

    if choice not in ["1", "2"]:
        print("Invalid Choice!")
        continue

    print("\n1. Encrypt")
    print("2. Decrypt")
    operation = input("Choose Operation: ")

    text = input("Enter Message: ")
    key = int(input("Enter Key: "))

    if choice == "1":
        if operation == "1":
            print("Encrypted Text:", rail_encrypt(text, key))
```


T125 SUMEET

```
        elif operation == "2":
            print("Decrypted Text:", rail_decrypt(text, key))
        else:
            print("Invalid Operation!")

    elif choice == "2":
        if operation == "1":
            print("Encrypted Text:", column_encrypt(text, key))
        elif operation == "2":
            print("Decrypted Text:", column_decrypt(text, key))
        else:
            print("Invalid Operation!")
```

OUTPUT:

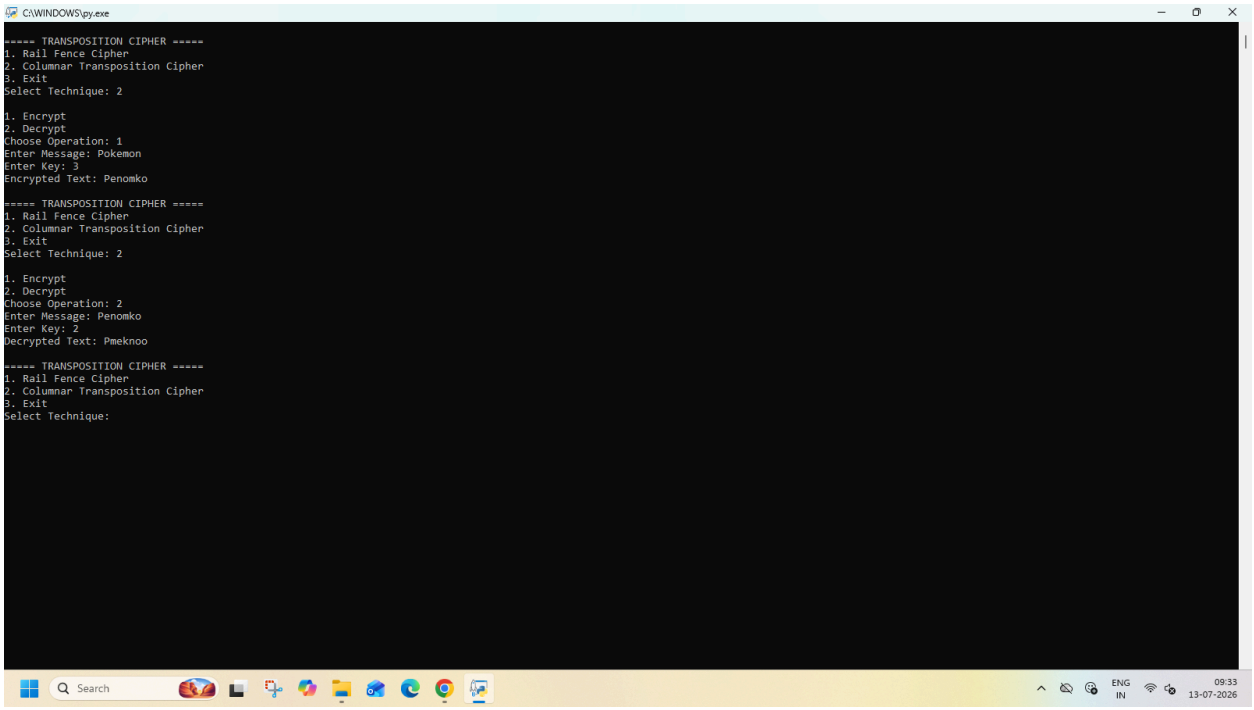
RailFence Cipher:



SHETH L.U.J AND SIR M.V. COLLEGE

SUBJECT: CYBER SECURITY

Columnar Transposition Cipher:



T125 - SU

SUMEET YADAV

T125

GUI:

CODE:

```
import tkinter as tk
from tkinter import ttk, messagebox
import math

#Rail Fence
def rail_encrypt(text, key):
    if key <= 1 or key >= len(text):
        return text

    rail = [['\n' for _ in range(len(text))] for _ in range(key)]

    row = 0
    direction = False

    for col in range(len(text)):
        if row == 0 or row == key - 1:
            direction = not direction

        rail[row][col] = text[col]

        if direction:
            row += 1
        else:
            row -= 1

    cipher = ""

    for i in range(key):
        for j in range(len(text)):
            if rail[i][j] != '\n':
                cipher += rail[i][j]

    return cipher


def rail_decrypt(cipher, key):
    if key <= 1 or key >= len(cipher):
        return cipher

    rail = [['\n' for _ in range(len(cipher))] for _ in range(key)]

    row = 0
```


T125_SUMEET

```
    direction = None

    for col in range(len(cipher)):
        if row == 0:
            direction = True
        if row == key - 1:
            direction = False

        rail[row][col] = '*'

        if direction:
            row += 1
        else:
            row -= 1

    index = 0

    for i in range(key):
        for j in range(len(cipher)):
            if rail[i][j] == '*' and index < len(cipher):
                rail[i][j] = cipher[index]
                index += 1

    result = ""
    row = 0

    for col in range(len(cipher)):
        if row == 0:
            direction = True
        if row == key - 1:
            direction = False

        result += rail[row][col]

        if direction:
            row += 1
        else:
            row -= 1

    return result


#Columnar
def column_encrypt(text, key):
```


T125 _ SUMEET

```
    if key <= 1 or key >= len(text):
        return text

    cipher = [''] * key

    for col in range(key):
        pointer = col
        while pointer < len(text):
            cipher[col] += text[pointer]
            pointer += key

    return ''.join(cipher)


def column_decrypt(cipher, key):
    if key <= 1 or key >= len(cipher):
        return cipher

    columns = math.ceil(len(cipher) / key)
    rows = key
    shaded = columns * rows - len(cipher)

    plain = [''] * columns

    col = 0
    row = 0

    for symbol in cipher:
        plain[col] += symbol
        col += 1

        if col == columns or (col == columns - 1 and row >= rows - shaded):
            col = 0
            row += 1

    return ''.join(plain)


#GUI
def encrypt():
    try:
        text = message_entry.get()
        key = int(key_entry.get())
```

T125 - SUMEET

```
        if key < 2 or key >= len(text):
            messagebox.showerror("Error", "Key must be between 2 and message length - 1")
            return

        if technique.get() == "Rail Fence":
            output.set(rail_encrypt(text, key))
        else:
            output.set(column_encrypt(text, key))

    except:
        messagebox.showerror("Error", "Invalid Key")


def decrypt():
    try:
        text = message_entry.get()
        key = int(key_entry.get())

        if key < 2 or key >= len(text):
            messagebox.showerror("Error", "Key must be between 2 and message length - 1")
            return

        if technique.get() == "Rail Fence":
            output.set(rail_decrypt(text, key))
        else:
            output.set(column_decrypt(text, key))

    except:
        messagebox.showerror("Error", "Invalid Key")


root = tk.Tk()
root.title("Transposition Cipher")
root.geometry("420x320")
root.resizable(False, False)

tk.Label(root, text="Transposition Cipher",
         font=("Arial", 16, "bold")).pack(pady=10)

tk.Label(root, text="Message").pack()
message_entry = tk.Entry(root, width=40)
message_entry.pack()

tk.Label(root, text="Key").pack()
```

SUMEET YADAV
T125

T125 - SUMEET

```
key_entry = tk.Entry(root, width=10)
key_entry.pack()

tk.Label(root, text="Technique").pack()

technique = ttk.Combobox(
   root,
   values=["Rail Fence", "Columnar"],
   state="readonly",
   width=20
)
technique.current(0)
technique.pack(pady=5)

tk.Button(root,
        text="Encrypt",
        width=15,
        command=encrypt).pack(pady=5)

tk.Button(root,
        text="Decrypt",
        width=15,
        command=decrypt).pack()

output = tk.StringVar()

tk.Label(root, text="Result").pack(pady=10)
tk.Entry(root,
       textvariable=output,
       width=45,
       state="readonly").pack()

root.mainloop()
```

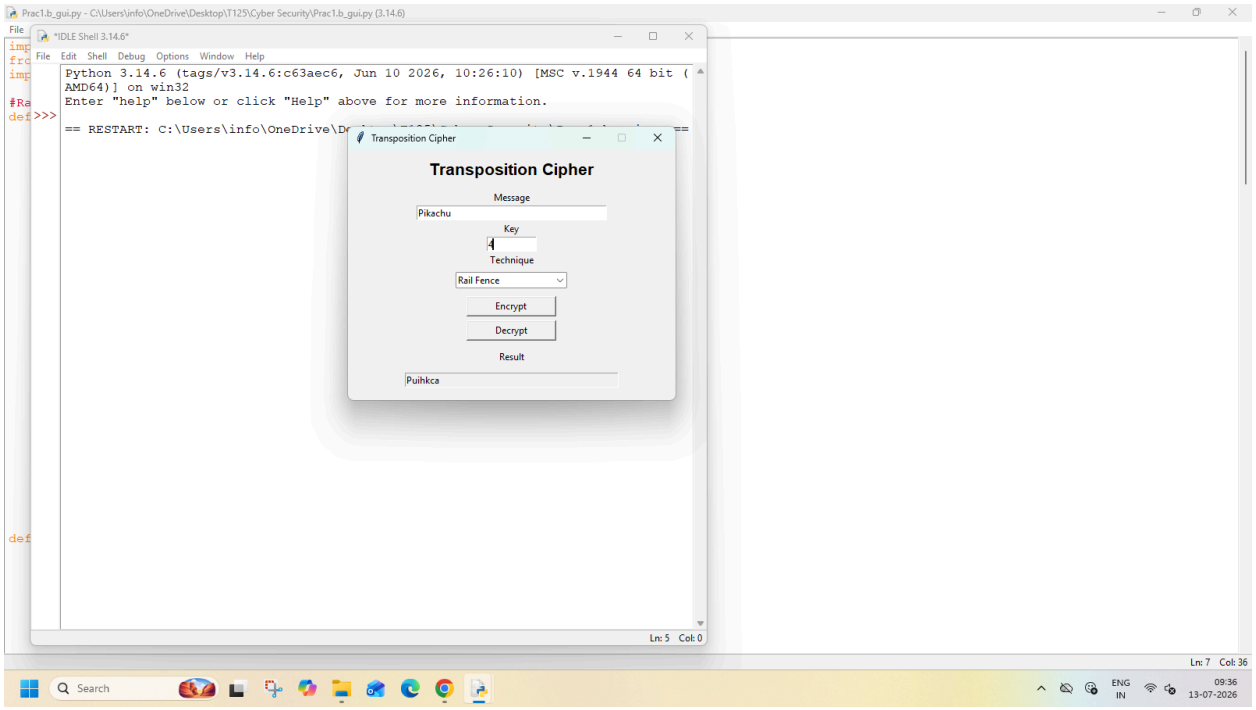
T125_SUMEET

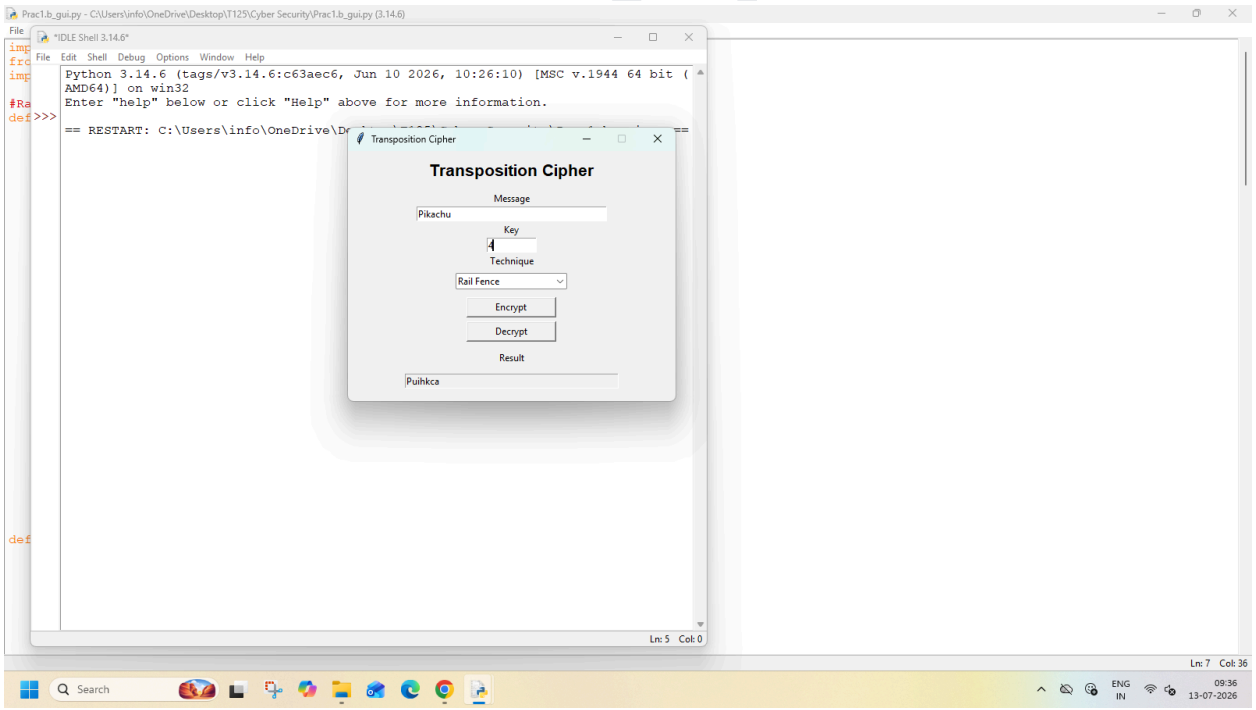
SHETH L.U.J AND SIR M.V. COLLEGE

SUBJECT: CYBER SECURITY

OUTPUT:

Rail Fence Cipher:





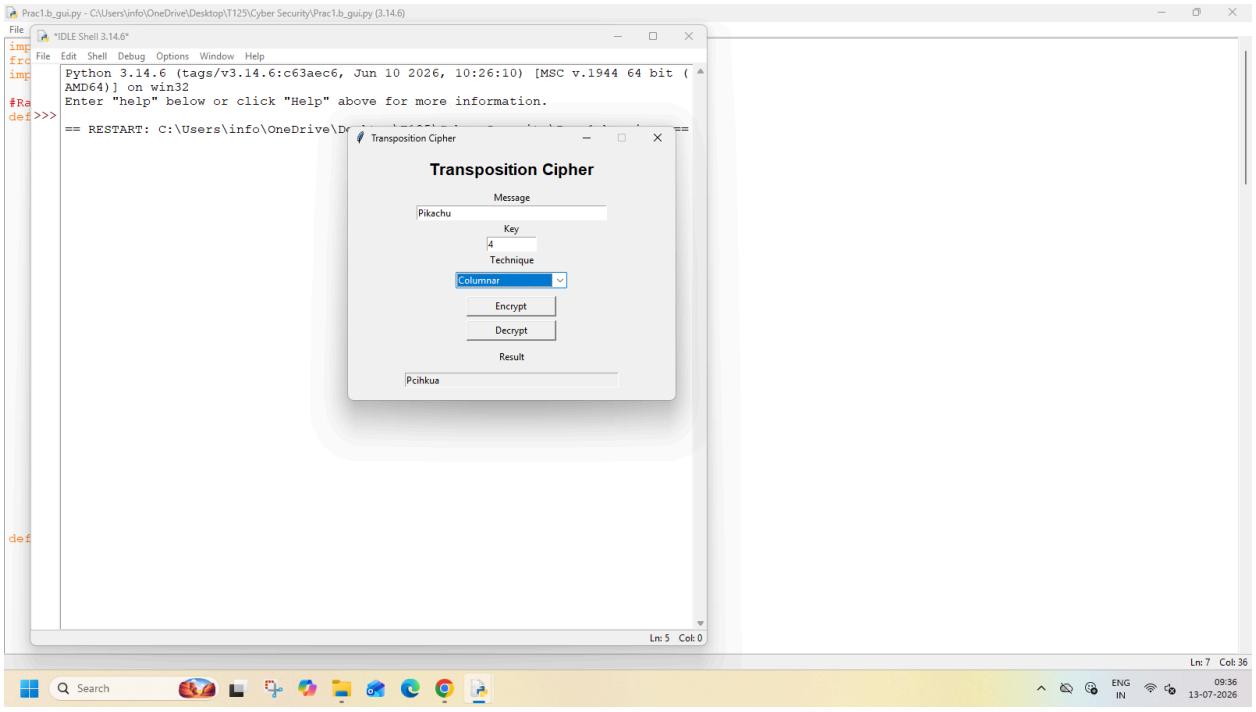
SUMEET YADAV

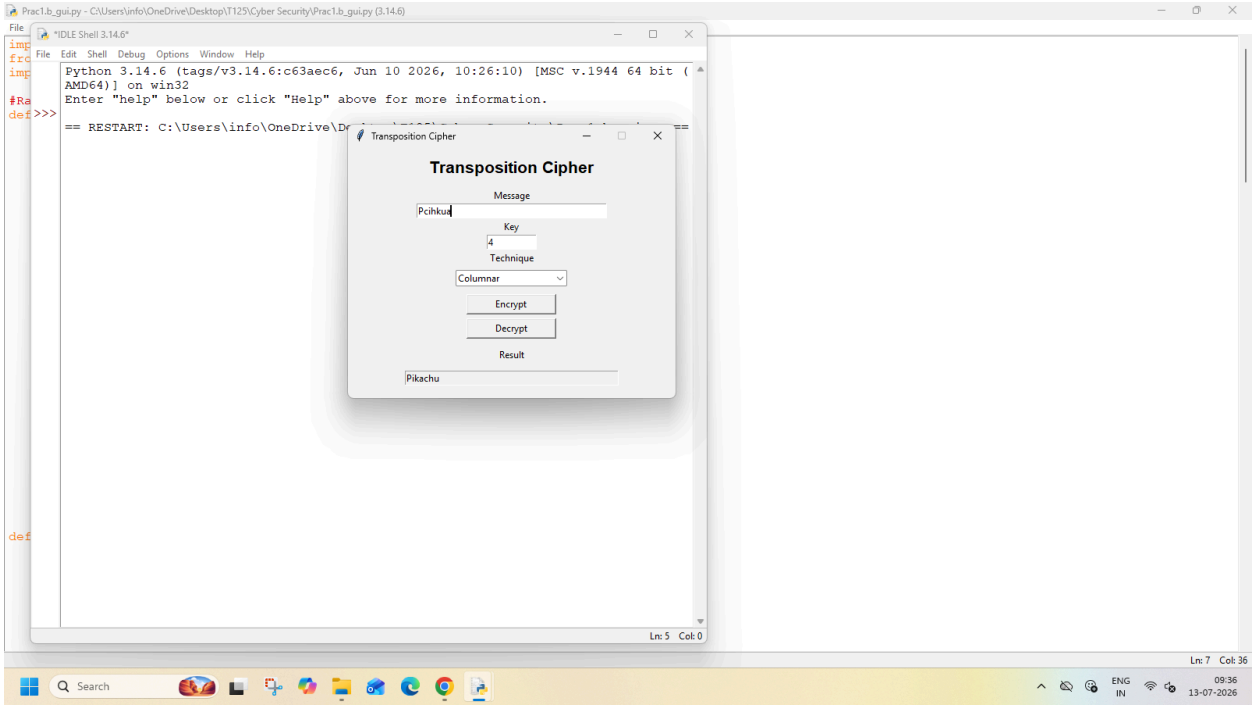
T125

SHETH L.U.J AND SIR M.V. COLLEGE

SUBJECT: CYBER SECURITY

Columnar:





SUMEET YADAV

T125